



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

**Taller investigativo: Métodos para seleccionar elementos del
DOM**

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Nicolas Ruiz Colorado**

Ficha: 3312932

Competencia:

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **29** de **agosto** del año **2026**



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

Tabla de Contenido

Métodos para seleccionar elementos del DOM	8
1. Document.getElementsByClassName	8
1.1. ¿Para qué sirve?	8
1.2. ¿Qué tipo de elementos permite seleccionar?.....	8
1.3. ¿Qué recibe como parámetro?	8
1.4. ¿Puede seleccionar uno o varios elementos?	8
1.5. ¿Qué devuelve?.....	8
1.6. ¿Como se puede acceder a los elementos encontrados?.....	8
1.7. ¿Cómo se pueden recorrer los elementos?	8
1.8. ¿Qué ocurre cuando no encuentra ningún elemento?	9
1.9. ¿Qué ventajas tiene?	9
1.10. ¿Qué limitaciones tiene?.....	9
2. document.getElementsByTagName()	9
2.1. ¿Para qué sirve?	9
2.2. ¿Qué tipos de elementos permite seleccionar?	9



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

2.3.	¿Qué recibe como parámetro?	9
2.4.	¿Puede seleccionar uno o varios elementos?	10
2.5.	¿Qué devuelve?.....	10
2.6.	¿Cómo se puede acceder a los elementos encontrados?	10
2.7.	¿Cómo se pueden recorrer?	10
2.8.	¿Qué ocurre cuando no encuentra ningún elemento?	10
2.9.	¿Qué ventajas tiene?	10
2.10.	¿ Que limitaciones tiene?	10
3.	Document.querySelector().....	11
3.1.	¿Para qué sirve?	11
3.2.	¿Qué tipo de elementos permite seleccionar?.....	11
3.3.	¿Qué recibe como parámetro?	11
3.4.	¿Puede seleccionar uno o varios elementos?	11
3.5.	¿Qué devuelve?.....	11
3.6.	¿Cómo se puede acceder al elemento encontrado?	11
3.7.	¿Cómo se pueden recorrer los elementos?	12
3.8.	¿Qué ocurre cuando no encuentra ningún elemento?	12
3.9.	¿Qué ventajas tiene?	12



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

3.10.	¿ Que limitaciones tiene?.....	12
4.	Document.querySelectorALL().....	12
4.1.	¿Para qué sirve?	12
4.2.	¿Qué tipo de elementos permite seleccionar?.....	12
4.3.	¿Qué recibe como parámetro?	13
4.4.	¿Puede seleccionar uno o varios elementos?	13
4.5.	¿Qué devuelve?.....	13
4.6.	¿Cómo se puede acceder a los elementos encontrados?	13
4.7.	¿Cómo se puede recorrer?	13
4.8.	¿Qué ocurre cuando no encuentra ningún elemento?	13
4.9.	¿Qué ventajas tiene?	13
4.10.	¿Qué limitaciones tiene?.....	14
5.	Comparaciones	14
5.1.	getElementById() vs getElementsByClassName()	14
5.2.	getElementsByClassName() vs getElementsByTagName().....	14
5.3.	querySelector() vs querySelectorALL()	15
5.4.	getElementsByClassName() vs querySelectorALL()	15
5.5.	getElementById() vs querySelector()	15



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

6.	Investigación sobre selectores CSS	15
6.1.	Selector por ID.....	16
6.2.	Selector por clase.....	16
6.3.	Selector por etiqueta	16
6.4.	Selector por atributo	16
6.5.	Selector descendiente	17
6.6.	Selector que tienen varias clases	17
6.7.	¿Por qué querySelector() y querySelectorAll() permiten realizar selecciones más específicas?	17
7.	Laboratorio de experimentación.....	17
7.1.	Código HTML	17
7.2.	Experimento 1.....	19
7.3.	Experimento 2.....	19
7.4.	Experimento 3.....	20
7.5.	Experimento 4.....	21
7.6.	Experimento 5.....	22
8.	Preguntas de Análisis.....	23



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

8.1.	¿Cuál es la diferencia entre seleccionar por ID y seleccionar por clase?	23
8.2.	¿Qué método utilizarías cuando necesitas seleccionar muchos elementos?	23
8.3.	¿Qué diferencia encuentras entre <code>querySelector()</code> y <code>querySelectorAll()</code> ?	23
8.4.	¿Qué método te parece más flexible? ¿Por qué?	23
8.5.	¿Qué método te parece más sencillo? ¿Por qué?	24
8.6.	¿Todos los métodos devuelven el mismo tipo de resultado?	24
8.7.	¿Qué sucede si no existe el elemento que estás buscando?	24
8.8.	¿Por qué es importante conocer los selectores CSS para trabajar con JavaScript?	24
8.9.	¿En qué situación utilizarías <code>getElementsByTagName()</code> ?	25
8.10.	¿En qué situación preferirías <code>querySelectorAll()</code> ?	25
9.	Reto Investigativo	25
9.1.	Respuesta	25
10.	Reto Practico	26
10.1.	Encontrar un elemento específico	26



**PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS**

10.2.	Encontrar varios elementos.	26
10.3.	Encontrar elementos por clase.	27
10.4.	Encontrar elementos por etiqueta.	27
10.5.	Realizar una selección más específica utilizando selectores.	28



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Métodos para seleccionar elementos del DOM

1. `Document.getElementsByClassName`

1.1. *¿Para qué sirve?*

Sirve para seleccionar elementos de HTML donde contengan una o más clases CSS.

1.2. *¿Qué tipo de elementos permite seleccionar?*

Permite seleccionar cualquier elemento del HTML que tenga una clase.

1.3. *¿Qué recibe como parámetro?*

Recibe el nombre de una clase, no es necesario escribir el punto como los usamos en CSS.

1.4. *¿Puede seleccionar uno o varios elementos?*

Se puede seleccionar uno o varios elementos que tengan esa clase

1.5. *¿Qué devuelve?*

Devuelve una colección de elementos llamada **HTMLCollection**

1.6. *¿Como se puede acceder a los elementos encontrados?*

Se puede acceder utilizando la posición del elemento, empezando desde el [0].

1.7. *¿Cómo se pueden recorrer los elementos?*

Se pueden recorrer utilizando el ciclo for



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

1.8. ¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve una colección vacía **HTMLCollection[]**

1.9. ¿Qué ventajas tiene?

- Permite encontrar varios elementos rápidamente
- Es útil cuando varios elementos comparten una misma clase }

1.10. ¿Qué limitaciones tiene?

- Solo permite buscar principalmente por clases
- Devuelve un HTMLCollection, que tiene algunas diferencias con un arreglo normal

2. document.getElementsByTagName()

2.1. ¿Para qué sirve?

Sirve para buscar elementos utilizando el nombre de su etiqueta **HTML < > </ >**

2.2. ¿Qué tipos de elementos permite seleccionar?

Puede seleccionar cualquier elemento utilizando su etiqueta HTML

2.3. ¿Qué recibe como parámetro?

Recibe el nombre de la etiqueta



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

2.4. ¿Puede seleccionar uno o varios elementos?

Puede encontrar uno o varios elementos que tengan esa etiqueta

2.5. ¿Qué devuelve?

Devuelve una colección **HTMLCollection**

2.6. ¿Cómo se puede acceder a los elementos encontrados?

Se puede acceder utilizando la posición del elemento []

2.7. ¿Cómo se pueden recorrer?

Se pueden recorrer con utilizando un ciclo for.

2.8. ¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve una colección vacía **HTMLCollection[]**

2.9. ¿Qué ventajas tiene?

- Es fácil de usar
- Es útil cuando queremos seleccionar todos los elementos de una etiqueta
- Permite trabajar con muchos elementos al mismo tiempo

2.10. ¿Que limitaciones tiene?

- La búsqueda puede ser muy general
- No permite hacer selecciones especificas como los selectores CSS



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

- No permite indicar directamente condiciones como una clase específica o atributo.

3. Document.querySelector()

3.1. *¿Para qué sirve?*

Sirve para buscar un elemento utilizando un selector CSS, además es bastante flexible porque permite utilizar ID, Clases, etiquetas, atributos y combinación de estos.

3.2. *¿Qué tipo de elementos permite seleccionar?*

Se puede seleccionar cualquier elemento de HTML que puede ser representado mediante un selector CSS.

3.3. *¿Qué recibe como parámetro?*

Recibe un selector CSS escrito como texto

3.4. *¿Puede seleccionar uno o varios elementos?*

Puede encontrar varios elementos que coincidan con el selector.

3.5. *¿Qué devuelve?*

Devuelve el primer elemento que coincide con el selector

3.6. *¿Cómo se puede acceder al elemento encontrado?*

Como devuelve directamente un elemento, podemos trabajar con el.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

3.7. ¿Cómo se pueden recorrer los elementos?

Normalmente no se utiliza para recorrer varios elementos porque solamente devuelve uno.

3.8. ¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve null

3.9. ¿Qué ventajas tiene?

- Es muy flexible
- Permite usar selectores CSS
- Permite hacer búsquedas bastante específicas
- Es útil cuando necesitamos un solo elemento

3.10. ¿Que limitaciones tiene?

- Si existen varios elementos que coinciden, solamente devuelve el primero
- Hay que conocer la sintaxis de los selectores CSS

4. Document.querySelectorALL()

4.1. ¿Para qué sirve?

Sirve para buscar todos los elementos que coincidan con un selector CSS

4.2. ¿Qué tipo de elementos permite seleccionar?

Puede seleccionar cualquier elemento HTML que coincida con un selector CSS.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

4.3. ¿Qué recibe como parámetro?

Recibe un selector CSS

4.4. ¿Puede seleccionar uno o varios elementos?

Puede seleccionar uno, varios o ninguno.

4.5. ¿Qué devuelve?

Devuelve un NodeList

4.6. ¿Cómo se puede acceder a los elementos encontrados?

Se puede acceder a ellos utilizando su posición en el NodeList

4.7. ¿Cómo se puede recorrer?

Se puede recorrer con un ciclo for.

4.8. ¿Qué ocurre cuando no encuentra ningún elemento?

Devuelve un NodeList vacío.

4.9. ¿Qué ventajas tiene?

- Es muy flexible
- Permite utilizar todos los selectores CSS
- Permite seleccionar varios elementos
- Facilita realizar búsquedas específicas



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

4.10. *¿Qué limitaciones tiene?*

- Es necesario conocer los selectores CSS
- Devuelve un NodeList, no un arreglo normal.
- Para algunas operaciones puede ser necesario convertir la colección a un arreglo

5. Comparaciones

5.1. *getElementById() vs getElementsByClassName()*

La principal diferencia es que `getElementById()` busca elementos por medio de su ID, mientras que por el `getElementsByClassName()` se busca elementos por medio de una clase. El ID es un elemento único, mientras que una misma clase puede estar en muchos elementos.

Yo utilizaría `getElementById()` cuando necesite trabajar con un elemento específico

Y `getElementsByClassName()` cuando necesite trabajar con varios elementos que compartan una clase.

5.2. *getElementsByClassName() vs getElementsByTagName()*

La diferencia está principalmente en la forma de realizar la búsqueda

Si se busca por clases se utiliza el `getElementsByClassName()`, pero si se busca por etiqueta se utilizaría el `getElementsByTagName()`.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

5.3. *querySelector()* vs *querySelectorALL()*

Los dos utilizan selectores CSS, pero tienen una diferencia importante

querySelector() devuelve el primer elemento que encuentra

querySelectorALL() devuelve todos los elementos que coinciden

5.4. *getElementByClassName()* vs *querySelectorALL()*

Los dos pueden utilizarse para seleccionar varios elementos por clase.

El *getElementByClassName* se usa específicamente para las clases.

Mientras que el *querySelectorAll()* utiliza selectores CSS y se pueden hacer búsquedas más específicas.

5.5. *getElementById()* vs *querySelector()*

Ambos se utilizan para encontrar un elemento específico, se pueden encontrar el mismo elemento la diferencia radica en que el *getElementById* está diseñado para buscar por ID, mientras que el *querySelector* permite utilizar diferentes tipos de selectores CSS.

6. Investigación sobre selectores CSS

Son una forma de indicar que elementos HTML se quiere seleccionar, aunque normalmente se utilizan para aplicar estilos, en JavaScript se pueden utilizar para buscar los



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

elementos mediante `querySelector` y `querySelectorAll`, existen varios tipos de selectores como:

- Selector por ID
- Selector por Clase
- Selector por Etiqueta
- Selector por Atributo
- Selector Descendiente

6.1. Selector por ID

Se utiliza un `#` seguido del ID y busca un elemento que contenga el id correspondiente: Ejemplo `querySelector("#titulo")`

6.2. Selector por clase

Se utiliza `.` seguido del nombre de la clase y busca los elementos que tengan la clase correspondiente: Ejemplo `querySelectorAll(".producto")`

6.3. Selector por etiqueta

Se escribe directamente el nombre de la etiqueta y busca todos los valores de la etiqueta correspondiente: Ejemplo `querySelectorAll("p")`

6.4. Selector por atributo

Permite buscar elementos que tengan un atributo determinado. Ejemplo:
`querySelectorAll("[type='button']")`



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

6.5. *Selector descendiente*

Permite buscar un elemento que se encuentre dentro de otro y busca la etiqueta o elemento que este dentro de una clase Ejemplo: `querySelectorAll(".productos p")`

6.6. *Selector que tienen varias clases*

Permite buscar un elemento que tengan simultáneamente las clases seleccionadas Ejemplo: `querySelectorAll(".producto.destacado")`

6.7. *¿Por qué `querySelector()` y `querySelectorAll()` permiten realizar selecciones más específicas?*

Porque utilizan la sintaxis de los selectores CSS, esto permite combinar diferentes condiciones, con los selectores CSS podemos combinar ID, clases, etiquetas atributos y relaciones entre elementos. Ejemplo: `document.querySelector(".producto.destacado")` es mas específico que buscar `document.getElementsByClassName("producto")`

7. Laboratorio de experimentación

7.1. *Código HTML*

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Laboratorio DOM</title>
  <link rel="stylesheet" href="css/index.css">
</head>
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

```
<body>

  <h1 id="titulo">Mis videojuegos</h1>

  <p class="descripcion">
    Estos son algunos videojuegos que me gustan.
  </p>

  <p class="descripcion">
    Cada juego tiene diferentes características.
  </p>

  <div class="videojuegos">

    <div class="favorito">
      <h2>SnowRunner</h2>
      
      <button class="btn">Ver información</button>
    </div>

    <div class="videojuego">
      <h2>Need for Speed PayBack</h2>
      
      <button class="btn">Ver información</button>
    </div>

    <div class="videojuego">
      <h2>Need for Speed Heat</h2>
      
      <button class="btn">Ver información</button>
    </div>

    <div class="videojuego ">
      <h2>EA Sport FC</h2>
      
      <button class="btn">Ver información</button>
    </div>

  </div>
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

```
<p id="nota" class="nota">
  Estos juegos son mis favoritos actualmente, son los que mas he
jugado en el ultimo año
</p>

<script src="js/experimento1.js"></script>
<script src="js/experimento2.js"></script>
<script src="js/experimento3.js"></script>
<script src="js/experimento4.js"></script>
<script src="js/experimento5.js"></script>
</body>
</html>
```

7.2. Experimento 1

```
// Experimento 1
//Seleccionar un elemento especifico
let titulo =document.getElementById("titulo")
console.log(titulo)

let titulo1=document.querySelector("#titulo")
console.log(titulo1)
```

```
<h1 id="titulo">Mis videojuegos</h1>
<h1 id="titulo">Mis videojuegos</h1>

> | ctrl i to turn on code suggestions. Don't show again
```

7.3. Experimento 2

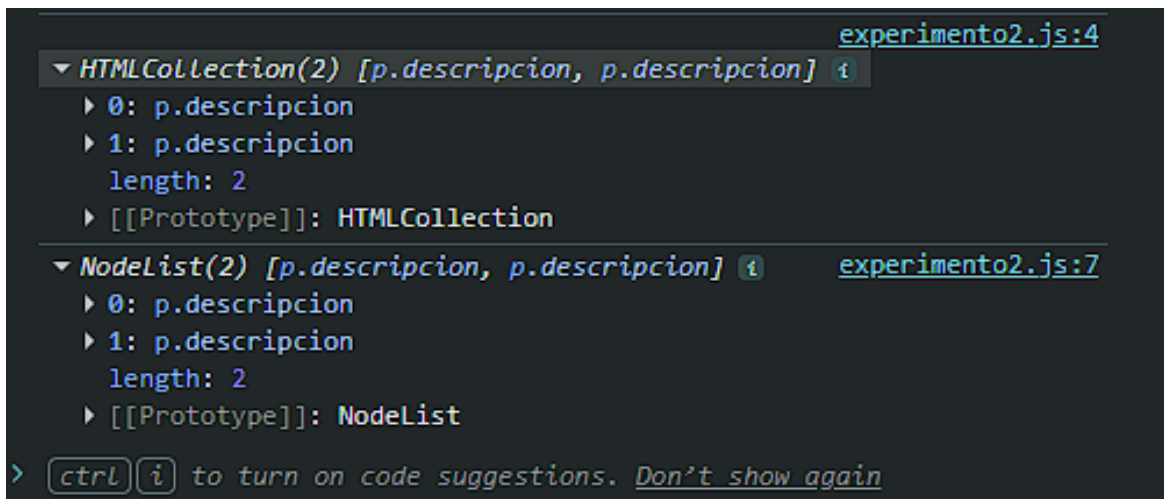
```
// Experimento 2
// Seleccionar varios elementos que compartan clase
let desc = document.getElementsByClassName("descripcion")
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

```
console.log(desc)

let descripciones = document.querySelectorAll(".descripcion")
console.log(descripciones)
```



7.4. Experimento 3

```
// Experimento 3
// Seleccionar todos los elementos de una determinada etiqueta
let imagen = document.getElementsByTagName("img")
console.log(imagen)

let image = document.querySelectorAll("img")
console.log(image)
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

```
▼ HTMLCollection(4) ⓘ experimento3.js:4
  ▶ 0: img
  ▶ 1: img
  ▶ 2: img
  ▶ 3: img
  length: 4
  ▶ [[Prototype]]: HTMLCollection

▼ NodeList(4) ⓘ experimento3.js:7
  ▶ 0: img
  ▶ 1: img
  ▶ 2: img
  ▶ 3: img
  length: 4
  ▶ [[Prototype]]: NodeList

Live reload enabled. index.html:87

> [ctrl][i] to turn on code suggestions. Don't show again
```

7.5. Experimento 4

```
// Experimento 4
// Seleccionar elementos utilizando diferentes tipos de selectores
let title = document.querySelectorAll("#titulo")
let clase = document.querySelectorAll(".favorito")
let etiqueta = document.querySelectorAll("img")
console.log(title)
console.log(clase)
console.log(etiqueta)
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

```
▼ NodeList [h1#titulo] ⓘ experimento4.js:6  
  ▶ 0: h1#titulo  
    length: 1  
  ▶ [[Prototype]]: NodeList  
▼ NodeList [div.favorito] ⓘ experimento4.js:7  
  ▶ 0: div.favorito  
    length: 1  
  ▶ [[Prototype]]: NodeList  
▼ NodeList(4) [img, img, img, img] ⓘ experimento4.js:8  
  ▶ 0: img  
  ▶ 1: img  
  ▶ 2: img  
  ▶ 3: img  
    length: 4  
  ▶ [[Prototype]]: NodeList  
> [ctrl][i] to turn on code suggestions. Don't show again
```

7.6. Experimento 5

```
// Experimento 5  
// Seleccionar un elemento que cumpla mas de una condicion  
let favorito = document.querySelectorAll(".videojuegos button")  
console.log(favorito)
```

```
experimento5.js:4  
▼ NodeList(4) [button.btn, button.btn, button.btn, button.btn] ⓘ  
  ▶ 0: button.btn  
  ▶ 1: button.btn  
  ▶ 2: button.btn  
  ▶ 3: button.btn  
    length: 4  
  ▶ [[Prototype]]: NodeList  
> [ctrl][i] to turn on code suggestions. Don't show again
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

8. Preguntas de Análisis

8.1. *¿Cuál es la diferencia entre seleccionar por ID y seleccionar por clase?*

Para mí la principal diferencia es que si tu seleccionas por id solo seleccionarías un elemento único es útil si quieres hacer un cambio o seleccionar un elemento específico, mientras que con la clase puedes seleccionar 1 o múltiples elementos es útil para realizar cambios en las clases del CSS o una selección de múltiples elementos.

8.2. *¿Qué método utilizarías cuando necesitas seleccionar muchos elementos?*

Si los elementos comparten una clase yo usaría `getElementByClassName`, pero si se requiere algo más específico utilizaría el `querySelectorAll`.

8.3. *¿Qué diferencia encontraste entre `querySelector()` y `querySelectorAll()`?*

La principal diferencia que encontré fue la cantidad de elementos que devuelve cada uno.

`querySelector()` devuelve únicamente el primer elemento que coincide, mientras que con `querySelectorAll` devuelve todos los elementos que coincidan.

8.4. *¿Qué método te parece más flexible? ¿Por qué?*

Según lo investigado el método más flexible sería el `querySelectorAll()` porque nos permite utilizar selectores CSS y seleccionar varios elementos al mismo tiempo.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

8.5. ¿Qué método te parece más sencillo? ¿Por qué?

Para mí el método mas sencillo de utilizar es getElementById porque solamente necesito conocer el ID del elemento que deseo seleccionar.

8.6. ¿Todos los métodos devuelven el mismo tipo de resultado?

No necesariamente los métodos getElementById, querySelector devuelven un elemento individual, los métodos getElementByClassName, getElementByTagName devuelven un HTMLCollection y querySelectorAll devuelve un NodeList.

8.7. ¿Qué sucede si no existe el elemento que estás buscando?

Depende del método que estemos utilizando, si el método que usamos es:

getElementById y el querySelector devuelven null; pero si el método es getElementByClassName, getElementByTagName y querySelectorAll devuelven una colección vacía.

8.8. ¿Por qué es importante conocer los selectores CSS para trabajar con JavaScript?

Creería que son importantes porque nos permite realizar búsquedas mas específicas dentro del HTML y también podemos aprovechar los mismos selectores desde JavaScript para encontrar elementos.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

8.9. ¿En qué situación utilizarías `getElementsByTagName()`?

Yo lo utilizaría cuando necesite obtener todos los elementos de una determinada etiqueta, por ejemplo, una imagen o un botón, para hacer una modificación o guardar los valores de estos elementos.

8.10. ¿En qué situación preferirías `querySelectorAll()`?

Según lo investigado creería que yo lo utilizaría cuando necesite seleccionar varios elementos, pero con condiciones específicas

9. Reto Investigativo

Un compañero afirma:

“Con `getElementById()` es suficiente para trabajar con el DOM, porque todos los elementos importantes pueden tener un ID.”

¿Estas de acuerdo?

Argumenta tu respuesta utilizando lo aprendido durante la investigación.

9.1. Respuesta

Posible si sería pero no sería lo más apropiado, porque este elemento se utiliza para encontrar un elemento específico; por ejemplo si hay 10 botones que realizan funciones similares sería más sencillo usar una misma clase para todos ellos, si usamos un `querySelectorAll` o un `getElementsByClassName` sería más práctico hacer una selección o un posible cambio.



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

Por lo tanto, considero que el `getElementById` es importante, pero este solamente es una de las diferentes herramientas que tenemos para trabajar con el DOM.

10. Reto Practico

Para este reto le solicite a la IA que me generará un HTML y CSS siguiendo las indicaciones del taller, para poder hacer los respectivos requerimientos:

10.1. *Encontrar un elemento específico.*

```
// Encontrar un elemento especifico
let titulo = document.getElementById("titulo-principal")
console.log(titulo)
```

```
<h1 id="titulo-principal">🎮 Catálogo de Videojuegos</h1> index.js:3
> [ctrl][i] to turn on code suggestions. Don't show again
```

10.2. *Encontrar varios elementos.*

```
//Encontrar varios elementos
let elementos = document.querySelectorAll(".categoria")
console.log(elementos)
```

```
index.js:7
▶ NodeList(3) [section#categoria-accion.categoria,
  section#categoria-aventura.categoria,
  section#categoria-estrategia.categoria]
> [ctrl][i] to turn on code suggestions. Don't show again
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL FORMATO ENTREGA DE EVIDENCIAS

10.3. Encontrar elementos por clase.

```
//Encontrar elementos por clase
let clases = document.getElementsByClassName("juego-titulo")
console.log(clases)
```

```
index.js:11
▶ HTMLCollection(8) [h3.juego-titulo, h3.juego-titulo,
  h3.juego-titulo, h3.juego-titulo, h3.juego-titulo, h3.juego-titulo,
  h3.juego-titulo, h3.juego-titulo]
> ⌘ i to turn on code suggestions. Don't show again
```

10.4. Encontrar elementos por etiqueta.

```
// Encontrar elementos por etiqueta
let etiqueta = document.getElementsByTagName("button")
console.log(etiqueta)
let etiq = document.querySelectorAll("button")
console.log(etiq)
```

```
index.js:15
▶ HTMLCollection(8) [button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar]
index.js:17
▶ NodeList(8) [button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar,
  button.btn.btn-agregar, button.btn.btn-agregar]
> ⌘ i to turn on code suggestions. Don't show again
```



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL
INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

10.5. *Realizar una selección más específica utilizando selectores.*

```
// Realizar una selección mas especifica utilizando selectores
let categorias = document.querySelectorAll(".categoria");

for (let categoria of categorias) {
  let ofertasEnCategoria = categoria.querySelectorAll(".juego.oferta");
  console.log(categoria.querySelector(".titulo-categoria"),
ofertasEnCategoria)
}
```

```
<h2 class="titulo-categoria">Acción</h2>                                index.js:24
  ▶ NodeList [article.juego.oferta]
<h2 class="titulo-categoria">Aventura</h2>                             index.js:24
  ▶ NodeList [article.juego.oferta]
<h2 class="titulo-categoria">Estrategia</h2>                           index.js:24
  ▶ NodeList [article.juego.oferta]
> (ctrl)(i) to turn on code suggestions. Don't show again
```